Lecture 03: Classes, Interfaces, Inheritance & Polymorphism

Objectives:

- Learn more about how C# programs are organized
- Learn how to declare Methods, Classes and Interfaces
- Learn how Inheritance and Polymorphism are achieved

In C#, the concept of a class, an interface, inheritance and polymorphism are very similar to what obtains in Java. Therefore, in this lecture, we shall concentrate on explaining the differences in these concepts between the two languages.

1. Organizing Types (Classes, Interfaces, Structs, Enums, etc.)

A C# program is a collection of types, defined in source files, organized by namespaces, and complied into **assemblies** (.exe or .dll files).

Generally, these organizational units overlap: a source file can contain many namespaces and a namespace can span several source files.

Similarly, an assembly can contain several namespaces and a namespace can spread across several assemblies.

For simplicity, Unless you have too many classes in a namespace, I suggest you put related classes into a single namespace, in a single source file and compile it into a single assembly.

2. Classes

As in Java, a class is declared using the *class* keyword.

A class can contain fields, constructors, methods and inner classes (only static inner classes)

In addition, a class in C# can contain *properties* and *events* (also indexers and operators).

Fields and methods may either be instance (default) or static.

Example 1:

```
using System;

namespace BankAccount {
     public class BankAccount {
          const double charityRate = 2.5;
          static int count;

          string name;
          int accountNumber;
          double balance;

          public BankAccount(string name) {
               this.name = name;
               accountNumber = ++count;
          }

          public BankAccount(string name, double amount):
this(name){
               balance = amount;
          }

          public void Deposit(double amount) {
               if (amount > 0)
                    balance += amount;
          }
          public void Withdraw(double amount) {
               if (balance >= amount) {
                    balance -= amount;
               }
          }
          public double GetBalance() {
               return balance;
          }
          public double GetAnnaulCharity() {
               double charity = balance * charityRate /100;
               balance -= charity;
               return charity;
          }
          public static void PrintCustomerCount() {
               Console.WriteLine("Number of Customers = "+count);
          }
          public override String ToString() {
               return "Acc #:"+accountNumber + ":"+name + ":
"+balance;
          }
     }


     class TestAccount {
          public static void Main() {
               BankAccount acc1 = new BankAccount("Sami", 2000);
               BankAccount acc2 = new BankAccount("Omar");
```

```
                acc1.Deposit(3000);
                acc1.Withdraw(4000);
                Console.WriteLine(acc1);

                acc2.Deposit(5000);
                acc2.Withdraw(2000);
                Console.WriteLine(acc2);


      BankAccount.PrintCustomerCount();
            }
      }
}
```

```
Acc #:1 Name:Sami Balance: 1000
Acc #:2 Name:Omar Balance: 3000
Number of Customers = 2
Press any key to continue . . .
```

Some notes from the example:

Constants:

Notice that the keyword *const* is used to define a constant.

Constants are evaluated at compile-time and are implicitly static.

The *this* keyword:

Notice that the *this* keyword is exactly as in Java. A reference to the current object. It is used for exactly the same two main purposes:

1. resolving name conflict between instance variables and method or constructor parameters.

2. Calling another constructor from another constructor in the same class. However, we note that the call is placed in the header of the calling constructor.

Access Modifiers:

To achieve encapsulation, a type may hide itself from other types or other assemblies by adding one of the following access modifies:

`public`	Fully accessible to all other types. This is the implicit accessibility for interfaces and enums.
`internal`	Accessible within the same assembly. This is the default for non-nested classes.
`private`	Accessible only from the type itself. This is the

	default for members of classes (and structs)
`protected`	Accessible within same class or sub-classes.
`protected internal`	Accessible by sub-classes as well as internal classes (equivalent to Java's protected).

Note:
A type or type member cannot be declared to be more accessible than any type it uses.

For example, a class cannot be *public* if it extends an *internal* class.

A method cannot be *protected* if the type of one of its parameters is *internal*.

Also access modifiers cannot be used when they conflict with the purpose of inheritance.

For example, an abstract method cannot be private.

Similarly, a **sealed** class cannot define new protected members since no class can benefit from them.

3. Abstract Classes

A class can be declared as abstract, using the *abstract* keyword.

An abstract class may have abstract methods as well as implemented methods.

Example 2:

```
using System;

public abstract class Shape {
      public String name() {
            return GetType().Name;
      }
      public abstract double Area();
      public abstract double Perimeter();

      public override String ToString() {
      return "ShapeType:"+name() + ":" + Perimeter() + ":" + Area();
      }
}
```

4. Interfaces

Like Java, Interfaces are used to minimize the effect of lack of multiple inheritance.

Interfaces contain only method specification without implementation. The methods are implicitly public and abstract – declaring them as such is an error.

Unlike Java, interfaces cannot have even constant fields.

A class can implement multiple interfaces. However, there is no "implements" keyword. Instead, a colon is used for implements.

Example 3:

```
using System;

public interface MyComparable {
     int CompareTo(Object obj);
}

public abstract class Shape : MyComparable {
     public String name() {
          return GetType().Name;
     }
     public abstract double Area();
     public abstract double Perimeter();

     public override String ToString() {
     return "ShapeType:"+name() + ":" + Perimeter() + ":" + Area();
     }
     public int CompareTo(Object obj) {
          Shape shape = (Shape) obj;
          if (Area()< shape.Area())
               return -1;
          else if (Area() > shape.Area())
               return 1;
          else
               return 0;
     }
}
```

5. Inheritance & Polymorphism

To achieve code re-usability, a class can inherit from another class – only single inheritance is allowed.

Again there is no "extend" keyword. Instead, the same colon

used for "implements" is used.

If a class extends a class and implements one or more interfaces, there should be only one colon. The super class and the interfaces are then listed separated by commas.

Notice that if there is a super class being extended, then it must appear first in the list.

The *base* keyword:

The keyword, ***base***, is used instead of the Java's ***super***, to refer to a superclass member.

Also it is used to call the constructor of the base class from within a subclass. However, like *this* keyword, such a call should be in the heading of the calling constructor.

Example:

```
class B:A {
    public B : base(. . .) {
        . . .
    }
}
```

Overriding & Hiding:

In C#, overriding is not allowed by default.

The base class must indicate that it is willing to allow its method to be overridden by declaring the method as **virtual**, **abstract** or **override**.

The subclass must also indicate that it is **overriding** the method by using the **override** keyword.

The effect of overriding is the same as in Java – Polymorphism. At run-time, a method call will be bound to the method of the actual object.

A subclass may also decide to **hide** an inherited method instead of overriding it by using the **new** keyword as the following example shows.

Example 4:

```
using System;

class A {
      public virtual void method() {
            Console.WriteLine(" In A");
      }
}
class B : A {
      public override void method() {
            Console.WriteLine("In B");
      }
}
class C : B {
      public new void method() {
            Console.WriteLine("In C");
      }
}
class Test {
      public static void Main() {
            C c = new C();
            c.method();    // calls C's method

            B b = c;
            b.method();  //calls B's method

            A a = c;
            a.method();  //calls B's method
      }
}
```

```
In C
In B
In B
Press any key to continue . . .
```

Properties:

Properties are the normal get and set methods we have in Java. In C#, the set and get operations are unified into a single unit.

Note that properties behave exactly like methods. They are inherited by subclasses and they can be hidden or overridden. Accordingly, they can have any of the modifiers that a normal method can have.

To define a property, you must have at lest one of get or set blocks.

Notice that compiler automatically defines a variable, *value*, in the set block to receive the set argument.

The following example put all the concepts together.

Example 5:

```
using System;
namespace Shapes {
      public abstract class Shape : IComparable {
            public String name() {
                  return GetType().Name;
            }
            public abstract double Area();
            public abstract double Perimeter();

            public override String ToString() {
                  return "ShapeType:"+name() + ":" + Perimeter() + ":"
+ Area();
            }
            public int CompareTo(Object obj) {
                  Shape shape = (Shape) obj;
                  if (Area()< shape.Area())
                        return -1;
                  else if (Area() > shape.Area())
                        return 1;
                  else
                        return 0;
            }
      }

      public class Rectangle : Shape {
            private double length;
            private double width;

            public double Length {
                  get {return length;}
                  set {length = value;}
            }
            public double Width {
                  get {return width;}
                  set {width = value;}
            }
            public Rectangle(double length, double width) {
                  this.length = length;
                  this.width = width;
            }
            public override double Area() {
             return length*width;
            }
            public override double Perimeter() {
            return 2*length + 2*width;
            }
      }

      public class Square : Rectangle {
```

```
        public Square(double length) : base(length, length) {
        }
    }
    public class Circle : Shape {
        private double radius;

        public double Radius {
            get {return radius;}
            set {radius = value;}
        }
        public Circle(double r) {
        radius = r;
        }
        public override double Area() {
        return Math.PI * (radius * radius);
        }
        public override double Perimeter() {
        return 2.0 * Math.PI * radius;
        }
    }

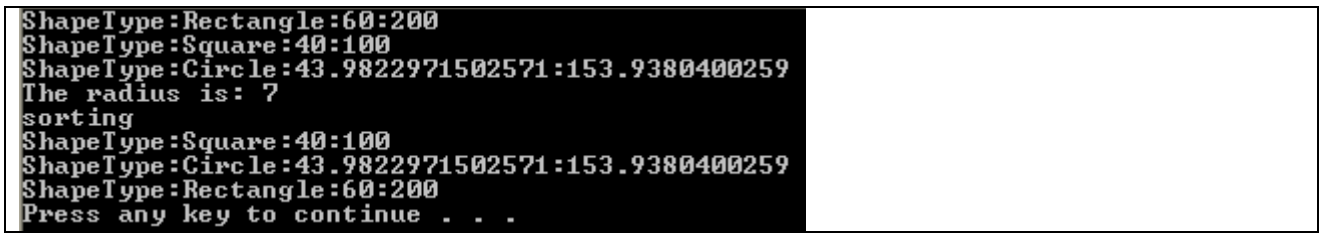
    public class TestShapes {
        public static void Main(String[] args) {
            Shape[] shape = new Shape[3];

            shape[0] = new Rectangle(20, 10);
            shape[1] = new Square(10);
            shape[2] = new Circle(7);

            for (int i=0; i<shape.Length; i++)
                Console.WriteLine(shape[i]);

            //using is and as operators
            foreach (Shape s in shape) {
                if (s is Circle) {
                    Circle c = s as Circle;
                    Console.WriteLine("The radius is:
"+c.Radius);
                }
            }

            //sorting the shapes
            Array.Sort(shape);
            Console.WriteLine("sorting");
            for (int i=0; i<shape.Length; i++)
                Console.WriteLine(shape[i]);
        }
    }
}
```


ShapeType:Rectangle:60:200
ShapeType:Square:40:100
ShapeType:Circle:43.9822971502571:153.9380400259
The radius is: 7
sorting
ShapeType:Square:40:100
ShapeType:Circle:43.9822971502571:153.9380400259
ShapeType:Rectangle:60:200
Press any key to continue . . .